

Politecnico di Torino

Politecnico di Torino

Master of science program in CYBERSECURITY

A.a. 2025/2026

Graduation Session March 2026

An Experimental Framework for Evaluating Secure Deletion Techniques in Modern Digital Environments

Relatori:

Andrea Atzeni

Giuseppe Emiliano Vaciago

Candidati:

Imesha Ann Anjalie Ponnamperumage

Abstract

Acknowledgements

Table of Contents

List of Tables VI

List of Figures VII

1 Introduction 1
- 1.1 NAND flash storage architecture . . . 2
  - 1.1.1 Cell Structure and Memory Organization . . . 2
  - 1.1.2 Basic Operations and Timing . . . 3
  - 1.1.3 Write-Once Property and Out-of-Place Updates . . . 3
  - 1.1.4 Flash Translation Layer . . . 3
- 1.2 Implications for Secure Deletion . . . 4

2 Systematic Review of Existing Literature 6
- 2.1 A Taxonomy of Anti-Forensic Techniques . . . 6
- 2.2 Technical Foundations of Secure Deletion . . . 7
  - 2.2.1 Core Concepts and Systematization . . . 7
  - 2.2.2 Secure Deletion Techniques . . . 8
  - 2.2.3 Flash Memory and SSD Challenges . . . 11
- 2.3 Legal and Ethical Dimensions . . . 12
- 2.4 Summary and Identified Research Gaps . . . 12

3 Systematic Review of Secure Deletion Methods 13

4 Experimental Framework and Methodology 14

5 Experimental Results and Analysis 15

6 Interpretation of Findings and Implications 16

7 Conclusions and Future Work 17

A Appendix A 18

Bibliography 19

List of Tables

2.1 Summary of Anti-Forensic Techniques 7

List of Figures

Chapter 1

Introduction

The task of deleting data and ensuring that they remain irrecoverable is known as Secure Data deletion. It has currently become one of the most pressing concerns in the cybersecurity, regulatory compliance, and digital forensics domains. While there exists physical means of making data irrecoverable in the physical world where confidential documents are shredded, and storage media containing classified material are physically destroyed, in the digital domain this is significantly complex. The standard delete operation on modern systems only implements file unlinking, which only changes the file system metadata while full contents will still remain intact in the system but users assume that all the content has been permanently erased. [1]

The criticality of this problem has been addressed by two intersecting forces. First, regulatory frameworks including GDPR right to erasure, NIST SP 800-88 Rev. 1, and IEEE 2883-2022 have mandated demonstrative and verifiable data removal evidences at defined sanitization levels; Clear, Purge and Destroy, with specific assurance against data recovery. Next, the mainstream transition from the usage of Hard Disk Drives (HDDs) to Solid-State Drives (SSDs) has led to a fundamental alteration of the storage landscape resulting in decades of secure deletion practices being inadequate.[2]

In contrast to HDDs, newer SSDs use a Flash Translation Layer (FTL) that maps logical block addresses to physical flash pages. The FTL performs out-of-place updates due to the erase-before-write property of NAND flash memory: modified data is written to a new physical location, and the old page containing the original data is marked as invalid. Furthermore, wear-leveling algorithms disperse data across the device to distribute program/erase cycles, and invalid pages are reclaimed by garbage-collection mechanisms only when free blocks run low, freeing them for new write operations. [2]

The problem of secure deletion not only occurs in the technical domain but also spans into the legal and ethical domains. On the legal front, regulations such

as the GDPR, the California Consumer Privacy Act (CCPA), and emerging AI governance frameworks impose obligations for data erasure that existing technical mechanisms struggle to satisfy.

The literature review below surveys state-of-the-art techniques in secure deletion, identifies methodological gaps, and motivates the need for a systematic framework that can reliably assess deletion effectiveness across diverse modern storage environments. The remainder of this thesis is organized as follows. Chapter 2 reviews the background of NAND flash storage architecture and the secure deletion problem in detail. Chapter 3 presents a systematic review of existing secure deletion techniques and their empirical evaluations. Chapter 4 describes the proposed experimental framework, including its metric system, test methodology, and verification protocols. Chapter 5 presents the results of applying the framework across a diverse set of storage devices and deletion techniques. Chapter 6 discusses the implications of the findings, Chapter 7 examines the legal and regulatory framework: the GDPR right to erasure, the right to be forgotten in emerging contexts, digital evidence and cross-border procedure, and broader data rights.and Chapter 8 concludes with directions for future work.

1.1 NAND flash storage architecture

NAND flash memory which was introduced by Toshiba in 1988, is a type of non-volatile semiconductor memory that has become the dominant storage technology for solid-state drives (SSDs), USB flash drives, and mobile devices [2]. Unlike NOR flash, which supports random access and high read performance at smaller capacities, NAND flash offers significantly higher storage density and faster write/erase operations at a lower cost, making it ideal for bulk data storage [3]

1.1.1 Cell Structure and Memory Organization

At the physical level, each NAND flash memory cell is a floating-gate transistor. Data is stored by injecting electrical charge into the floating gate, which is electrically isolated and can retain charge without power. A charged floating gate (programmed state) represents a logical "0," while an uncharged gate (erased state) represents a logical "1" [3]. Single-level cell (SLC) flash stores one bit per cell, whereas multi-level cell (MLC) stores two bits, triple-level cell (TLC) stores three bits, and quad-level cell (QLC) stores four bits. Each increase in bit density reduces cost per gigabyte at the expense of reduced endurance and lower performance [2].

Memory cells are arranged hierarchically. Cells connected to the same word line form a page, which is the smallest unit for read and program (write) operations. Pages are grouped into blocks, the smallest unit for erase operations. Blocks are organized into planes, multiple planes reside on a single die, and multiple dies are

packaged into a NAND chip. Typical page sizes range from 2 KB to 16 KB, and blocks typically contain 64 to 256 pages[2]. Each page also includes a spare or out-of-band (OOB) area used to store error-correcting codes (ECC) and metadata [4].

1.1.2 Basic Operations and Timing

The three fundamental operations on NAND flash are read, program, and erase. Read and program operate at page granularity, while erase operates at block granularity. Their latencies differ by orders of magnitude: a page read takes approximately 20 to 60 μs, a page program takes 200 to 500 μs μs, and a block erase takes 1.5 to 5 ms [2, 4]

1.1.3 Write-Once Property and Out-of-Place Updates

NAND flash has a fundamental constraint: a page cannot be overwritten in place. To write new data to a page, the entire encompassing block must first be erased, which resets all cells in that block to the "1" (erased) state [2] Because erasing a block is an expensive operation that destroys all data within it, SSDs do not perform in-place updates. Instead, they use out-of-place updates: modified data is written to an available free page elsewhere, and the old page is marked as invalid [5]. Pages containing current data are termed valid pages, while those holding superseded data are invalid pages. This write-once property is the root cause of many of the secure deletion challenges discussed in this review.

1.1.4 Flash Translation Layer

To maintain compatibility with traditional block-based file systems (e.g., ext4, NTFS), SSDs incorporate a Flash Translation Layer (FTL) that runs on the device controller [2]. The FTL maps logical block addresses (LBAs) issued by the operating system to physical flash addresses. This abstraction hides the complexity of NAND flash from the host and enables the SSD to manage internal operations transparently [4].

The FTL implements several critical management functions:

Wear leveling distributes program/erase (P/E) cycles evenly across all blocks to extend device lifespan. Each NAND flash block has a finite endurance typically 100,000 cycles for SLC and 10,000 to 100,000 for MLC/TLC after which the block becomes unreliable. Dynamic wear leveling distributes writes among free blocks, while static wear leveling periodically relocates infrequently accessed (cold) data to prevent some blocks from remaining idle while others wear out [3].

Garbage collection reclaims space occupied by invalid pages. When the number of free pages falls below a threshold, the FTL selects a victim block with a high proportion of invalid pages, copies any remaining valid pages to a free block, and then erases the victim block [2, 4]. This process introduces write amplification, where a single host write triggers multiple internal writes due to data migration.

Bad block management tracks blocks that have become defective through wear or manufacturing defects. These blocks are recorded in a bad block table and excluded from further use, with any valid data relocated to healthy blocks [4, 3].

Over-provisioning reserves a portion of the total flash capacity as spare area to improve performance and endurance. This over-provisioned space is used for wear leveling, garbage collection, and bad block management. [2, 6]

Error correction is essential because NAND flash cells are susceptible to read disturb, program disturb, charge leakage, and retention errors. The FTL computes ECC data stored in each page's OOB area to detect and correct bit errors[5].

An SSD packages multiple NAND chips with a controller that manages the FTL, host interface, and error correction. The storage architecture spans several layers: the host interface (e.g., SATA or NVMe) communicates with the FTL, which translates LBAs to physical addresses and coordinates access across multiple channels, packages, and dies in parallel to maximize throughput. To overcome the density scaling limits of planar (2D) NAND, manufacturers have adopted 3D NAND technology, which stacks memory cells vertically in layers [2]. 3D NAND achieves higher storage density without shrinking individual cell geometries, which improves endurance and reduces interference between adjacent cells. However, the vertical architecture introduces more complex disturb characteristics and new challenges for secure deletion, as erasing data in one layer can disturb adjacent layers [6].

1.2 Implications for Secure Deletion

The architectural features described above create fundamental obstacles to secure deletion. Out-of-place updates cause multiple copies of data to persist across the device as stale pages are left behind. Garbage collection and wear leveling can relocate data transparently, making it impossible for host-level software to predict where data physically resides. Over-provisioned areas are inaccessible to the operating system, yet may retain residual copies of deleted data. These characteristics mean that a conventional delete command at the file-system level does not remove the underlying data from the physical medium, and techniques that are effective on HDDs such as overwriting are ineffective on NAND flash [2].

Achieving secure deletion in modern systems is fundamentally challenging because storage architectures are organized into multiple layers application, file system,

block device, and physical medium each of which can retain residual copies of data [4]. Performing deletion at the application layer, for example, only removes the logical reference; the underlying data remains in the file system and may persist on the physical media indefinitely [5].

Chapter 2

Systematic Review of Existing Literature

The growing dependence on digital systems for both personal and professional activities has resulted in an unprecedented accumulation of sensitive information. As organizations and individuals increasingly store confidential data on a wide range of digital media—ranging from traditional hard drives to solid-state drives and cloud environments—the need for reliable and verifiable data deletion mechanisms has become critical. Failure to ensure proper data sanitization can expose users to privacy breaches, corporate espionage, and regulatory non-compliance. From a cybersecurity perspective, secure deletion plays a dual role. On one hand, it is an essential component of data protection and privacy preservation, ensuring that information is irretrievably removed when no longer needed. On the other hand, it can also be employed as part of anti-forensic strategies, deliberately obstructing digital investigations by erasing potential evidence. Understanding this duality is fundamental to appreciating the technical and ethical challenges that surround secure deletion.

2.1 A Taxonomy of Anti-Forensic Techniques

The field of digital forensics is founded on the assumption that traces of user activity persist within digital systems and can be extracted for analysis and attribution. Anti-Forensics challenges this assumption by allowing the deliberate use of various techniques, procedures, tools and stratergies to obstruct the collection of digital evidences from a computer system. The main objective of these methodologies are to hinder Data Collection and impede Forensic investigations. These techniques aim to obscure, alter, or destroy digital evidence in ways that reduce the likelihood of recovery or interpretation. González Arias, et al. [7] provide one of the most

comprehensive systematic reviews of this domain, analyzing 220 studies spanning from 2006 to 2022. Their taxonomy identifies four overarching categories of anti-forensic activity: **data hiding, artefact deletion, obfuscation of traces, and attacks on forensic tools and methods.**

Table 2.1: Summary of Anti-Forensic Techniques

Category	**What it does**	**Examples**
Data Hiding	Conceals data so it's difficult to find, while keeping it accessible to the adversary	Encryption, steganography, filesystem manipulation, Host Protected Area modification
Artifact Deletion	Permanently destroys or overwrites data to eliminate recoverable traces	Secure file deletion, disk wiping utilities, degaussing, physical destruction
Obfuscation of traces	Confuses or misdirects the forensic process	Log cleaners, timestamp alteration, spoofing, backbone hopping
Attacks Against Tools/Methods	Directly targets forensic software or procedures	Compression bombs, hash collision attacks, anti-reverse engineering

Secure deletion falls under the **artifact deletion** category, as it focuses on the permanent destruction of data to prevent recovery.

2.2 Technical Foundations of Secure Deletion

2.2.1 Core Concepts and Systematization

The secure deletion literature has produced a rich taxonomy of approaches operating at different system layers. Reardon et al. (2013) provide the most comprehensive systematization in their SoK paper, classifying secure deletion approaches by their interface to the physical medium — from user-level applications down to device drivers — and presenting a taxonomy of adversaries with varying capabilities [1]. They identify key behavioral properties that discriminate approaches: deletion latency, physical wear, and environmental assumptions about how the interfaces use affects the physical medium.

Joukov et al. (2006) address several persistent misconceptions about secure deletion. They argue that encryption is not an ideal solution for all contexts, that existing data-overwriting tools do not work as well as commonly believed, and that securely deleted files do not require many overwrite passes in most realistic scenarios [8]. Their work introduces important but often neglected issues such as

secure deletion consistency during power failures, the challenges posed by versioning and journaling file systems, and the necessity of overwriting metadata.

Peterson et al. present algorithms and an architecture for secure deletion of individual file versions in versioning file systems, motivated by federally compliant storage that must eliminate data after a mandatory retention period [9]. Their approach combines authenticated encryption with selective overwriting and demonstrates order-of-magnitude performance improvements over prior methods.

Wang et al. (2020) provide a comprehensive survey of secure deletion techniques for both HDDs and SSDs, with particular attention to emerging storage technologies such as shingled magnetic recording (SMR) and interlaced magnetic recording (IMR) HDDs [10]. Their work categorizes deletion approaches into encryption-based, erasure-based, and overwriting-based categories, and evaluates the pros and cons of implementing secure deletion at different system layers from FTL firmware modifications to file-system-level approaches. They observe that SMR and IMR HDDs impose sequential write constraints due to overlapping track structures, requiring new deletion techniques beyond traditional overwriting approaches designed for conventional magnetic recording (CMR) HDDs.

2.2.2 Secure Deletion Techniques

Aronsson et al. (2018) categorizes secure deletion methods into three primary approaches: **overwriting-based**, **encryption-based**, and **erasure-based**. Each approach operates at a different system layer and offers distinct trade-offs among security, performance, and device longevity [2].

Overwriting-Based Approaches

Overwriting-based methods replace existing data with new patterns (zeros, ones, or random data) to prevent recovery. On HDDs, techniques such as Gutmann's 35-pass overwriting have been widely adopted [9]. However, on SSDs, overwriting is fundamentally ineffective because the FTL maps overwritten logical blocks to new physical pages, leaving the original data intact on the old pages [2]. Scrubbing a technique that reprograms individual flash pages to destroy their data state attempts to address this at the physical level but introduces disturbance effects on adjacent cells and data errors, particularly in 3D NAND flash [11]. Empirical studies confirm that even after multiple overwrite passes, a significant portion of deleted data remains recoverable on SSDs [2].

Erasure-Based Approaches

Erasure-based methods physically remove data from the storage medium, typically by issuing block-erase commands or triggering garbage collection [4]. The primary

advantage of erasure-based deletion is that it provides physical removal of data without relying on cryptographic assumptions. However, these methods suffer from high performance costs because the basic deletion unit is a flash block, which requires migrating valid data from the target block before erasure [1]. The resulting write amplification reduces SSD lifespan. Advanced erasure-based designs include scrubbing-aware schemes that combine secure deletion operations with scrubbing to optimize performance by exploiting disturbance properties, achieving significant reductions in average deletion time [11].

Cryptographic Erasure Approaches

Encryption-based secure deletion, or cryptographic erasure, treats the storage medium as a black box. Data is encrypted before storage and rendered irrecoverable by securely deleting the encryption key, making the approach independent of the underlying hardware characteristics [12].

Onarlioglu et al. (2018) present Eraser, a system providing secure deletion guarantees at file granularity [12]. Eraser encrypts every file on an insecure medium with a unique key that can later be discarded. To make the approach scalable on commodity systems, keys are organized in an efficient tree structure where a single master key is confined to a secure store, implemented as a TPM. Eraser is deployed as a file-aware stackable block device — a standalone Linux kernel module requiring no modifications to the operating system — and demonstrates comparable overhead to existing full-disk encryption implementations.

Zhang et al. (2018) survey the combination of plausibly deniable encryption with secure deletion for end-to-end data confidentiality [5]. Their work frames the dual requirement: data must be protected during its lifetime (via encryption) and rendered irrecoverable after its lifetime (via secure deletion). The survey covers encryption-based deletion mechanisms across different system layers and storage media, identifying the trade-offs between security guarantees and performance overhead.

The evolution of key management architectures for cryptographic erasure has progressed through several generations. DNEFS (Data Node Encrypted File System) encrypts each data block with a unique key and colocates keys in a dedicated storage area, achieving secure deletion by propagating only the relevant keys [1]. Eraser improved scalability by organizing keys in a tree structure where a single master key stored in a TPM protects all file keys, enabling deletion through key revocation [12]. However, Eraser does not provide crash consistency, and post-crash experiments have shown that up to 64 files may be lost when a TPM master key rotation desynchronizes from the on-disk file key table [13].

Holepunch, proposed by Ratliff Wittmann et al., addresses this gap by providing memory-efficient, crash-consistent secure deletion using puncturable pseudorandom

functions (PPRFs). [13]. Holepunch treats the storage device as a black box and uses per-file keys to transparently encrypt outgoing writes and decrypt incoming reads, ensuring that all physical data on the device is always encrypted. To delete a file, Holepunch performs a puncture operation on the PPRF key, causing the pseudorandom function to permanently "forget" how to generate the associated file key. This approach eliminates the need to store or explicitly erase individual file keys, reducing both memory pressure and disk I/O compared to prior tree-based key management schemes. Holepunch stores its master key in TPM secure storage and uses a novel journaling scheme to provide crash consistency between TPM state and on-disk structures, ensuring that the file system remains usable after a power failure or software crash. Performance evaluations show that Holepunch provides similar I/O performance to Eraser (within negligible margins for small-file workloads) while consuming significantly less RAM as disk size grows, making it particularly suitable for large-capacity drives.

Integrated Approaches

Recognizing the limitations of individual methods, several works propose hybrid approaches that dynamically select deletion strategies. ErasuCrypto, proposed by Liu et al., integrates erasure-based and encryption-based methods by formulating deletion as a cost-minimization problem, dynamically choosing the most cost-effective technique based on the percentage of invalid pages in a block [10] The goal is to minimize data migrations and block erasures, thereby reducing performance degradation and extending SSD lifetime. Experimental results show that ErasuCrypto can smartly select between erasure and encryption approaches to reduce energy consumption and write amplification.

Building on ErasuCrypto, Li et al. propose a temperature-aware scheme that classifies data by hot/cold degree and allocates them to separate blocks [10]. Blocks with hot data can be deleted via erasure-based methods with minimal migration overhead, while cold-data blocks use the ErasuCrypto approach, effectively reducing secure deletion costs.

SAFE (Scramble and Finally Erase) takes a different approach, combining encryption with erasure to provide near-instantaneous secure deletion with verifiability at the controller level. SAFE encrypts all data with a controller-managed key. To sanitize the device, the controller key is erased, all blocks are erased, overwritten with a specific pattern, erased again, and then the device is re-initialized with a new key. This approach is simple, fast, and easy to verify because it exploits the advantages of both encryption-based and erasure-based approaches [10].

Additionally, erasure-based techniques such as purging and ballooning attempt to force garbage collection to delete invalid data: purging fills the storage with junk files to trigger garbage collection, while ballooning constrains free space to

invoke garbage collection more frequently [10]. The Fine-grained Overwriting-based Techniques approach, such as Lin et al.'s one-shot sanitization for MLC NAND, exploits the one-shot programming mechanism to sanitize pages at very low latency (less than 1% overhead) by carefully controlling threshold voltage distributions.

2.2.3 Flash Memory and SSD Challenges

SSDs compound the secure deletion problem through their flash translation layer (FTL), which maps logical block addresses to physical pages. Due to the erase-before-write property of NAND flash, the FTL performs out-of-place updates, leaving old physical pages containing stale data until garbage collection reclaims them. Wear leveling further disperses data across the device to distribute program-erase cycles, making it impossible for host-level software to predict where data physically resides [12].

Lee et al. design a NAND flash file system with secure deletion functionality, modifying YAFFS to support it [14]. Their method uses encryption and forces all keys of a specific file to be stored in the same block, requiring only one erase operation to securely delete a file along with all of its metadata. Results show low amortized block erasure counts compared to simple encryption methods.

Wang et al. (2020) survey secure deletion techniques designed for higher efficiency, verifiability, and portability across HDDs and flash-based SSDs [10]. They discuss the pros and cons of implementing secure deletion at different system layers — from FTL firmware modifications to file-system-level approaches — providing guidance for future storage system designs.

Chen and Chen (2022) identify a critical gap in flash-based secure deletion: duplicate data [4]. Even when the primary copy of a file is securely deleted, duplicate copies generated by FTL operations, garbage collection, and flash-friendly file systems may persist. They propose an approach that explicitly removes these duplicates, demonstrating that comprehensive deletion must account for the copy proliferation inherent in flash storage.

Aronsson and Lund conduct a survey of data erasure methods with a focus on SSDs and their unique challenges compared to HDDs. Their empirical measurements on commercial SSDs reveal inconsistencies in deletion times across vendors and device models. They find that combining cryptographic erasure with block erase is necessary for sufficient security, and that most existing deletion standards cover similar ground but lack specificity for SSD internals [2].

Homaidi (2009) provides an early framing of data remanence in SSDs, documenting how the physical characteristics of flash memory create persistent residual data risks that earlier deletion methods designed for magnetic media fail to address [3].

The challenges are further amplified by the transition to 3D NAND technology,

which stacks memory cells vertically in multiple layers to increase storage density. Wang et al. (2018) propose a scrubbing-aware secure (SAS) deletion design for 3D NAND flash that exploits scrubbing disturbance properties to improve deletion efficiency [11]. Scrubbing reprograms a flash page to destroy its original data state, providing page-granularity deletion finer than block erasure, but introduces disturbance effects on adjacent cells that become more severe in vertically stacked architectures [11]. The SAS design formulates overhead calculations for both scrubbing and erase operations, organizes sensitive data into scrubbing-friendly patterns via a grouping-based write strategy (GBWS), and exploits disturbance infection to accelerate scrubbing performance. Experimental results on a simulated 128 Gbit 3D NAND device demonstrate an 82% reduction in scrubbing latency and a 78% improvement in overall system response time [11].

2.3 Legal and Ethical Dimensions

2.4 Summary and Identified Research Gaps

Chapter 3

Systematic Review of Secure Deletion Methods

Chapter 4

Experimental Framework and Methodology

Chapter 5

Experimental Results and Analysis

Chapter 6

Interpretation of Findings and Implications

Chapter 7

Conclusions and Future Work

Appendix A

Appendix A

```
1     const test = test;
```

$\mathcal{O}\left(n \log n\right)$
`numpy`

Bibliography

[1] Joel Reardon, David Basin, and Srdjan Čapkun. «SoK: Secure Data Deletion». In: *Proceedings of the 2013 IEEE Symposium on Security and Privacy (SP'13)*. San Francisco, CA, USA: IEEE, 2013, pp. 301–315. DOI: 10.1109/SP.2013.28 (cit. on pp. 1, 7, 9).

[2] Fabian Aronsson and Oskar Lund. «Secure Data Deletion: Ensuring Confidentiality in Digital Systems ; A Survey of Methods for Data Erasure». LIU-IDA/LITH-EX-A–2025/074–SE. Master's thesis. Linköping, Sweden: Linköping University, 2025. URL: http://www.ep.liu.se/ (cit. on pp. 1–4, 8, 11).

[3] Omar A. Homaidi. «Data Remanence: Secure Deletion of Data in SSDs». Bachelor's thesis. Blekinge Institute of Technology, 2009. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:bth-5165 (cit. on pp. 2–4, 11).

[4] Niusen Chen and Bo Chen. «Duplicates also Matter! Towards Secure Deletion on Flash-based Storage Media by Removing Duplicates». In: *Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security*. ASIA CCS '22. Nagasaki, Japan: Association for Computing Machinery, 2022, pp. 54–66. ISBN: 9781450391405. DOI: 10.1145/3488932.3523255. URL: https://doi.org/10.1145/3488932.3523255 (cit. on pp. 3–5, 8, 11).

[5] Qi Zhang, Siqi Jia, Bo Chang, Yan Chen, and Zheli Liu. «Ensuring data confidentiality via plausibly deniable encryption and secure deletion – a survey». In: *Cybersecurity* 1.1 (2018), p. 1. DOI: 10.1186/s42400-018-0005-8. URL: https://doi.org/10.1186/s42400-018-0005-8 (cit. on pp. 3–5, 9).

[6] Na Young Ahn and Dong Hoon Lee. «Forensics and Anti-Forensics of a NAND Flash Memory: From a Copy-Back Program Perspective». In: *IEEE Access* 9 (2021), pp. 14130–14137. DOI: 10.1109/ACCESS.2021.3052353 (cit. on p. 4).

[7] Rafael González Arias, Javier Bermejo Higuera, J. Javier Rainer Granados, Juan Ramón Bermejo Higuera, and Juan Antonio Sicilia Montalvo. «Systematic Review: Anti-Forensic Computer Techniques». In: *Applied Sciences*

14.12 (2024). ISSN: 2076-3417. DOI: 10.3390/app14125302. URL: https://www.mdpi.com/2076-3417/14/12/5302 (cit. on p. 6).

[8] Nikolai Joukov, Harry Papaxenopoulos, and Erez Zadok. «Secure deletion myths, issues, and solutions». In: *Proceedings of the Second ACM Workshop on Storage Security and Survivability*. StorageSS '06. Alexandria, Virginia, USA: Association for Computing Machinery, 2006, pp. 61–66. ISBN: 1595935525. DOI: 10.1145/1179559.1179571. URL: https://doi.org/10.1145/1179559.1179571 (cit. on p. 7).

[9] Zachary N. J. Peterson, Randal Burns, Joe Herring, Adam Stubblefield, and Aviel D. Rubin. «Secure Deletion for a Versioning File System». In: *Proceedings of the 4th USENIX Conference on File and Storage Technologies (FAST '05)*. San Francisco, CA, 2005 (cit. on p. 8).

[10] Wei-Chen Wang, Chien-Chung Ho, Yu-Ming Chang, and Yuan-Hao Chang. «Challenges and Designs for Secure Deletion in Storage Systems». In: *2020 Indo – Taiwan 2nd International Conference on Computing, Analytics and Networks (Indo-Taiwan ICAN)*. 2020, pp. 181–189. DOI: 10.1109/Indo-TaiwanICAN48429.2020.9181335 (cit. on pp. 8, 10, 11).

[11] Wei-Chen Wang, Chien-Chung Ho, Yuan-Hao Chang, Tei-Wei Kuo, and Ping-Hsien Lin. «Scrubbing-Aware Secure Deletion for 3-D NAND Flash». In: *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 37.11 (2018), pp. 2790–2801. DOI: 10.1109/TCAD.2018.2857260 (cit. on pp. 8, 9, 12).

[12] Kaan Onarlioglu, William Robertson, and Engin Kirda. *Eraser: Your Data Won't Be Back*. 2018. DOI: 10.1109/EuroSP.2018.00019 (cit. on pp. 9, 11).

[13] Zachary Ratliff, Wittmann Goh, Abe Wieland, James Mickens, and Ryan Williams. *Holepunch: Fast, Secure File Deletion with Crash Consistency*. Cryptology ePrint Archive, Paper 2023/1927. 2023. URL: https://eprint.iacr.org/2023/1927 (cit. on pp. 9, 10).

[14] Jongmoo Lee, Jihong Heo, Youngjae Cho, Joonwon Hong, and Seungyong Shin. «Secure Deletion for NAND Flash File System». In: *Proceedings of the 2008 ACM Symposium on Applied Computing (SAC'08)*. Fortaleza, Ceará, Brazil: ACM, 2008, pp. 1710–1714. ISBN: 978-1-59593-753-7. DOI: 10.1145/1363686.1364093 (cit. on p. 11).